

TafDB设计思路

海量数据对数据湖存储的扩展能力提出了极高的要求。元数据面作为云存储最核心、最底层的系统之一，直接决定了存储系统的扩展性。

本文作为数据湖系列的第二篇，将为大家揭开元数据面存储底座的秘密，如何设计能够支撑存储容量的“无限扩展”。

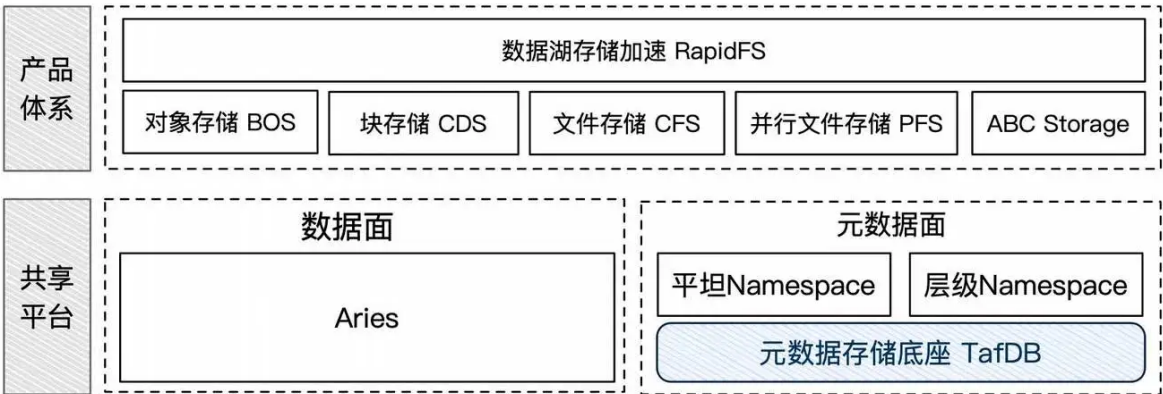
本文将底层的核心技术用通俗易懂的语言讲述出来，同时又不失专业性，不容错过。

当今企业面临数字化转型和智能化升级的挑战，作为承载了庞大算力的云基础设施，成为企业打破这种挑战的重要支撑。

随着移动互联网、物联网、AI 计算等技术和市场的迅速发展，数据规模指数级膨胀，IDC 预测全球数据量从 2018 年的 33 ZB 将会增长至 2025 年的 175 ZB，云存储系统的扩展性面临更大的挑战。

云存储系统一般由数据面和元数据面构成，其中数据面用于存储用户数据，元数据面用于存储数据对应的 meta 信息。用户数据量和访问量的增加会导致元数据面存储的条目数和 QPS 增加，元数据面的扩展性会直接影响到整个存储系统的扩展性。

TafDB 是[百度沧海·存储](#)的统一的元数据底座，支撑了百度智能云[对象存储](#)和[文件系统](#)的元数据存储，提供了万亿级别元数据规模、千万级别 QPS 的元数据存储能力，可以满足海量数据湖存储扩展性和性能的要求。



在介绍 TafDB 前我们先了解下元数据面技术的演进趋势。

1. 元数据面的技术演进

对象存储和文件存储的元数据面其实就是 Namespace，分为层级 Namespace 和平坦 Namesapce 两类。

文件存储操作的对象是文件和目录，Namespace 要支持目录树语义，元数据面基于层级 Namesapce 构建；对象存储主要操作对象是对象（Object），通常没有目录和文件夹的概念，元数据面基于平坦 Namespace 构建。

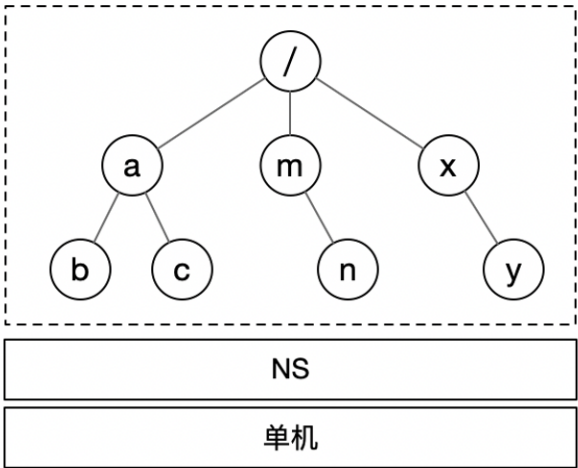
最近对象存储为了支持兼容 HDFS 语义的数据湖分析，也开始支持层级 Namespace。

2. 层级 Namespace 技术演进

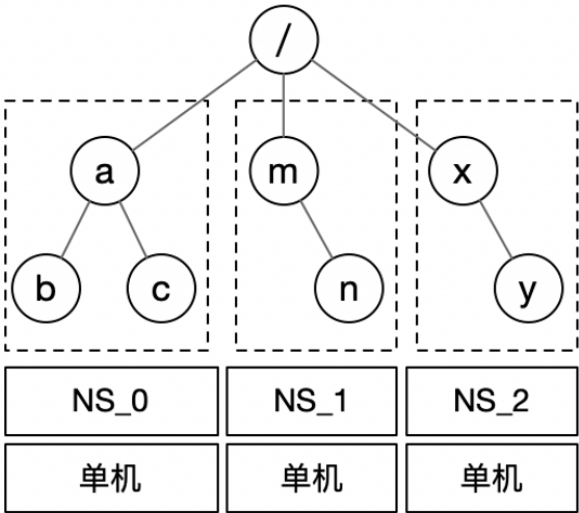
层级 Namespace 主要是维护文件系统的文件属性、目录树结构等元数据信息，同时支持目录树及文件操作，如：文件/目录创建、查找 (Lookup/Getattr) 删除及重命名 (Rename) 等。

当前，业界分布式文件系统领域衍生出各种类型的 Namespace 技术架构，可以归类为如下几种：

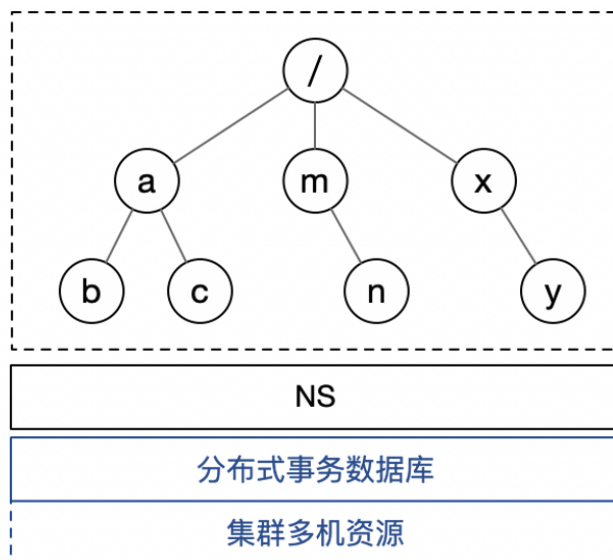
- 单机架构：该架构方案把所有目录树单机全内存，可做到低延迟，但是无法横向扩展，最大规模仅支持 10 亿文件数，代表产品为 HDFS。



- 基于子树划分：该架构方案通过将层级目录树拆分成多个子树，并将每颗子树按照相应的负载策略部署到不同的 Meta 节点中，但缺点是容易产生热点，负载均衡难以实现，扩展性不够好，同时难以做到跨子树的 Rename，典型的实现如 HDFS Federation、CephFS、IndexFS。

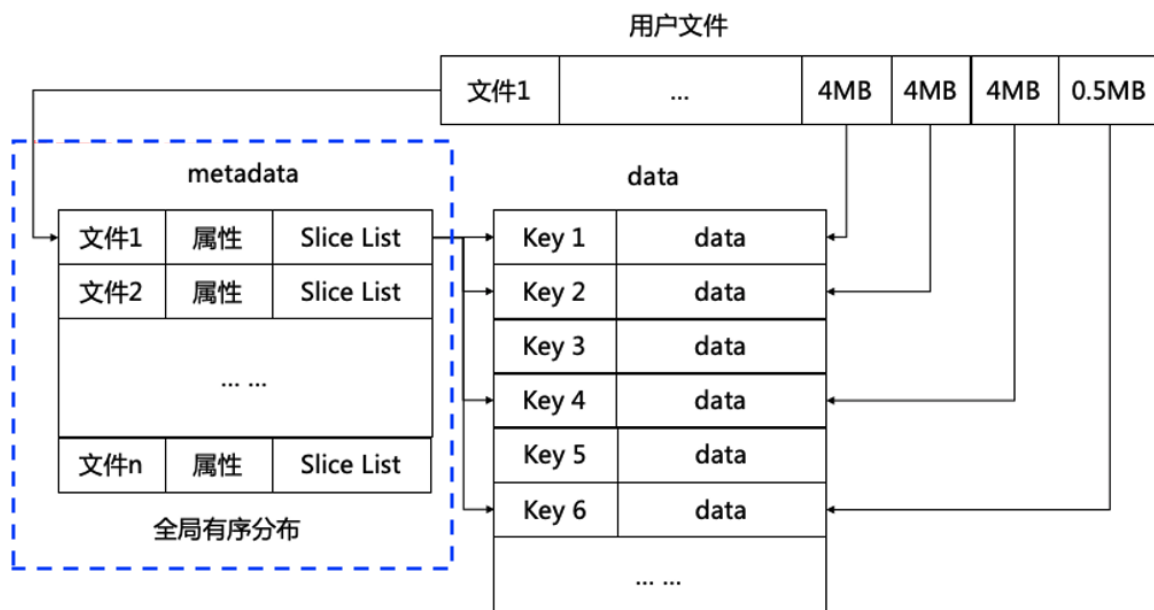


- 基于分布式事务数据库：上层维护了一层元数据语义层，该层将目录树操作转化为数据库事务请求。下层是分布式数据库，负责元数据的存储管理，目录树中的每个 inode 节点对应数据库中的一行记录。此方案可以做到单集群文件数规模无上限，这也是目前的技术趋势，典型的实现如 Facebook Tectonic。



3. 平坦 Namespace 技术演进

平坦 Namespace 主要存储一个对象的文件块的位置信息列表，一个对象文件分成了 N 个小块，每个小块文件都存储在数据面系统中，这个对象文件的块位置列表则存储在平坦 Namespace 中。逻辑结构如下图所示：



对象存储的元数据体量通常非常庞大，单机架构无法有效支撑。早期方案一般基于数据库中间件存储元数据，这类方案存在两个比较大的问题：1) 扩展性存在瓶颈——扩容只能倍扩，对成本造成很大压力；2) 对跨库的分布式事务支持不好。而目前的主流方案一般基于分布式事务数据库，这从根本上解决了数据库中间件的扩展性问题，代表产品为 AWS、Google 的对象存储，元数据分别存放在 Dynamodb 和 Spanner。



通过上面两类 Namespace 的技术演进趋势可以看到，使用分布式事务数据库可以完美解决元数据存储的扩展性问题，做到同时支持层级和平坦 Namespace，满足万亿级元数据底座要求。

4. 元数据底座的技术选型

在使用分布式事务数据库解决扩展性问题后，我们通过对层级和平坦 Namespace 业务场景的分析，发现其在功能和性能上还有如下需求：

- 完备的数据库功能：高性能点操作、范围操作、事务操作；分布式备份恢复、CDC（Change Data Capture）
- 极致的性能与扩展性：万亿级别元数据规模、千万级别 QPS

分布式事务数据库当前有这几个技术流派：

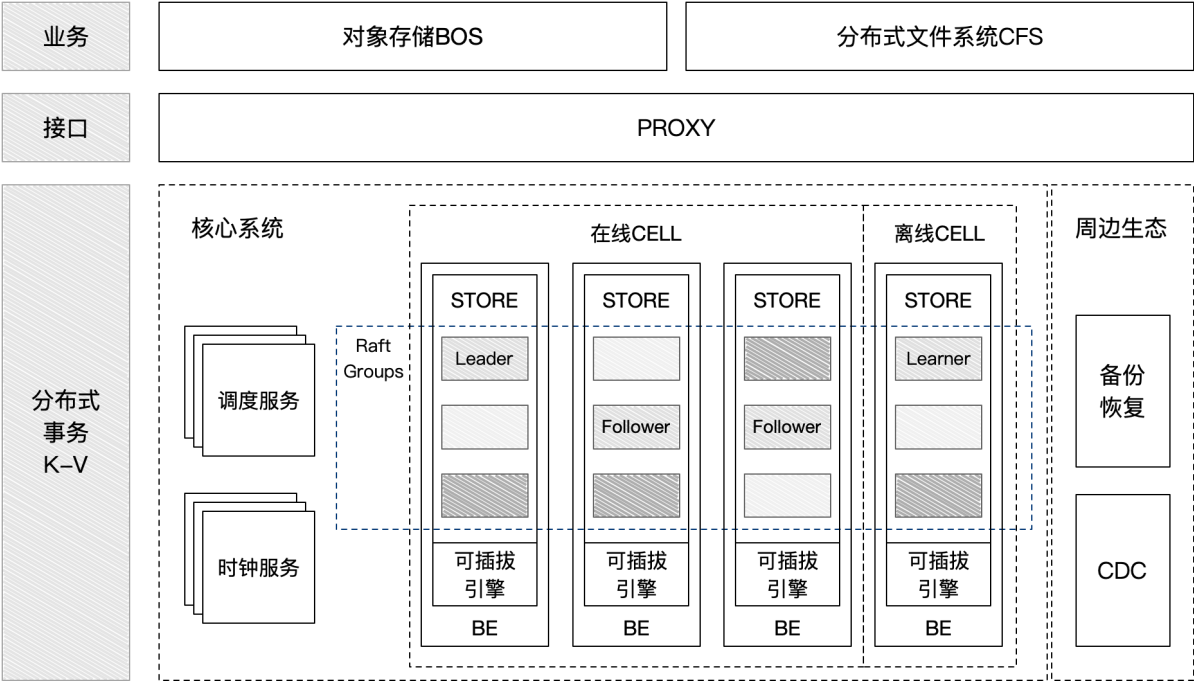
1. Spanner：全球部署、配备原子钟，并基于 2PC（两阶段提交）实现分布式强一致事务，具备极高的可用性和自动增容的扩展性。这种架构由 Google 设计研发，适合元数据存储这种对扩展性、可用性、性能有极高要求的场景。
2. Calvin：没有采用 2PC 实现，其目标是尽量减少分布式事务的开销。具体来说就是在事务执行以前就确定了所有操作的执行顺序。但 Calvin 不支持交互式事务，而在我们的场景中交互式事务的需求是存在的，比如说级目录树的 Rename 操作，因此它不满足我们的需求。
3. FoundationDB：苹果在 2018 年开源的分布式事务 K-V，它也是世界上第一个给 NoSQL 增加 ACID 能力的系统，FDB 采用微服务的理念把系统进行解耦，将事务管理系统与分布式存储分离，并独立地扩展它们。这种架构从软件工程角度确实做到了彻底解耦，但会导致写入路径增加多次 RPC 开销。同时由于系统内部存在广播，出现慢节点的概率变大，系统容易出现长尾，因此我们觉得不适合层级 Namespace 这种极低延迟场景。

在对业界的分布式事务数据库进行一系列调研后，我们发现 Calvin 和 FoundationDB 等在功能或性能方面不符合我们的需求，只有 Spanner 架构合适。虽然社区也已经存在针对通用场景的实现，但通用系统会因为通用性而牺牲性能。而且鉴于这是百度智能云存储的核心底座，需要自主可控并提供给最佳的用户体验。最终我们决定面向百度沧海·存储的元数据场景，自主研发一套类 Spanner 架构的分布式事务数据库作为元数据底座。

5. 百度智能云的云存储元数据底座 TafDB

TafDB 是面向元数据场景设计的一个分布式数据库系统。作为百度沧海·存储统一的元数据底座，TafDB 支撑了百度智能云对象存储 BOS 和文件存储 CFS 的元数据存储，提供了万亿级别元数据规模、千万级别 QPS 的元数据存储能力。

5.1 系统架构



TafDB 基于 RocksDB 实现单机存储，基于 Multi Raft 协议保障副本数据一致性，其中：

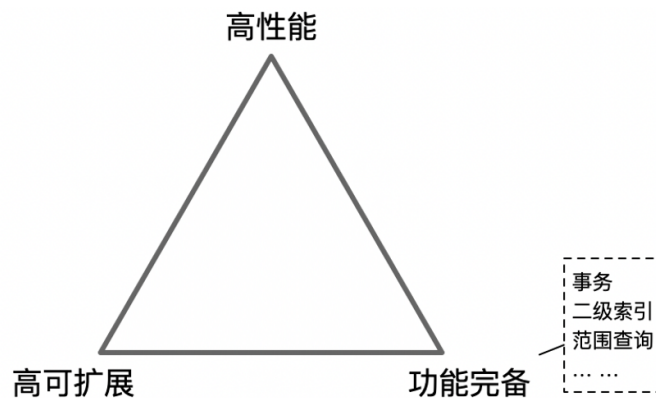
- BE：负责数据存储，用 Tablet 组织，不同 BE 的多个 Tablet 形成一个 Raft group 实现副本的高可用。BE 实例宕机可以自动补齐副本数据。
- 调度服务 Master：负责元信息管理，包含分区、容量、均衡等，基于 Raft 实现高可用。
- Proxy：负责前端 SQL 解析、事务协调、查询计划生成执行，无状态多实例部署。
- 时钟服务 TimeService：全局时钟服务，负责提供给单调递增的时钟服务，正在逐渐被新的分布式 TS 方案所替代。

5.2 系统特性

- 功能完备：支持全局有序、分布式事务、二级索引、分布式查询、分布式备份、CDC (Change Data Capture) 等。
- 高性能：面向元数据场景设计，元数据读写场景性能领先开源方案 2 倍+。
- 强扩展性：具备支撑万亿级元数据存储的能力。支持单集群 EB 级数据存储。

6. TafDB 的工程挑战

实现上述功能完备的分布式事务数据库系统本身就是一个巨大的挑战，同时在此基础上做到高性能和高可扩展性，则是难上加难。比如事务、索引等功能，通常在系统扩展到一定规模后会严重影响性能。



具体来说还要解决以下三个典型问题：

1. 在保证元数据操作 ACID 的同时，降低分布式事务的高额开销 —— 解决事务功能和系统性能的矛盾
2. 在提供高性能写操作的同时，保证范围查询的性能 —— 解决连续删除 + 范围查询和性能的矛盾
3. 消除数据流程的单点，提供极致的扩展性和可用性 —— 解决多版本（事务）功能和高扩展性的矛盾

6.1 挑战一：在保证元数据操作 ACID 的同时，降低分布式事务的高额开销

6.1.1 痛点

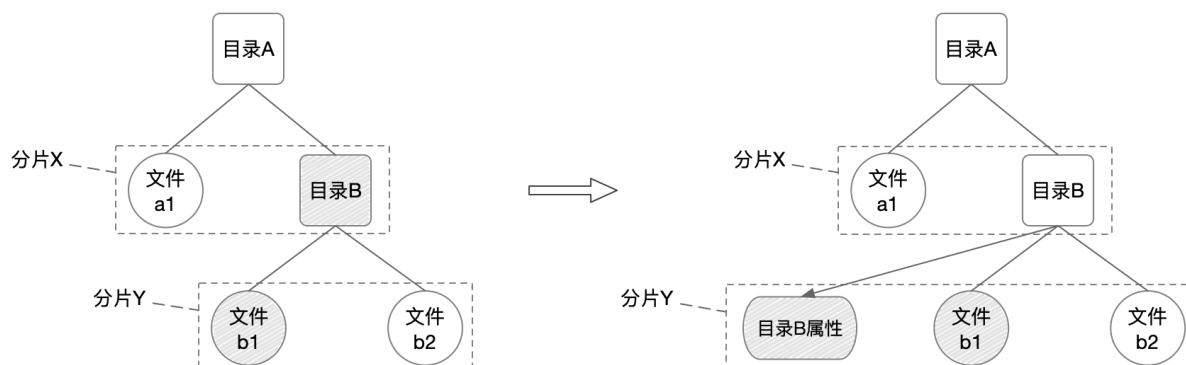
分布式系统对 ACID 的强要求，一般通过分布式事务来保证。如果要写入的多条记录在多个不同的分片上，类 Spanner 架构系统需要使用两阶段提交来完成分布式事务，以保证事务的原子性。但两阶段提交会涉及到多次的 RPC 交互，性能开销极大。而元数据存储场景中可能触发大量跨分片事务，比如：

- 文件系统层级 Namespace 存在大量事务语义，目录树操作中修改的节点通常在不同分片上。
- 对象存储平坦 Namespace 本身事务语义很少，但是业务需使用全局二级索引，主键数据和索引数据在不同的分片上。

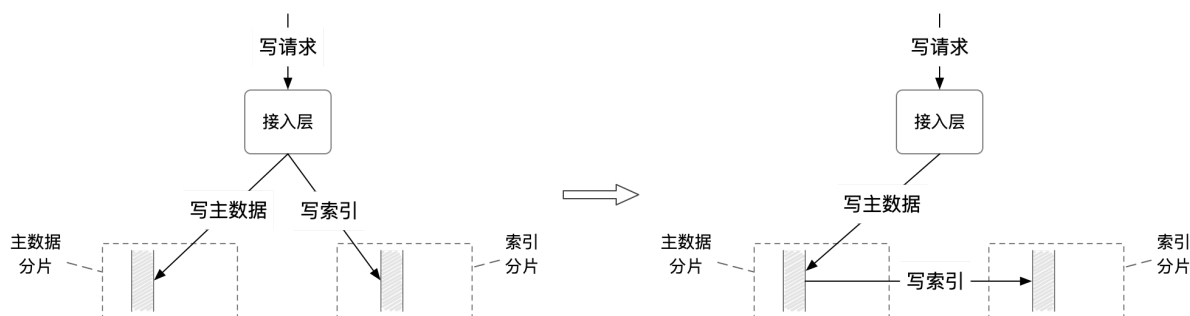
分布式事务过高的性能开销会导致不可接受的写入延迟。我们解决这个问题的主要思路就是将一个事务涉及的数据都集中在同一个分片，来消除系统中的跨分片事务。

6.1.2 解决方案

- 针对层级 Namespace：TafDB 提供了一种自定义分裂策略的功能，来保证同层目录元数据分片不分裂。同时业务调整表结构，控制一次目录树要操作的节点都在同层目录。这样就可以把绝大部分目录树操作都控制在单个分片上。如图所示，对文件 b1 操作涉及文件 b1 和其父目录 B 的元数据修改，原本需要通过分布式事务同时完成分片 X 和 Y 的数据写入。而优化后，我们将目录 B 需要修改的部分属性也放在分片 Y 中，这样只需写分片 Y 即可。



- 针对平坦 Namespace：我们分析发现大部分业务场景需要的二级索引不要求极致的时效性。基于这个特点我们实现了一种机制来异步写入索引数据，以略微降低索引时效性为代价，将绝大部分写入都控制在单个分片上。如图所示，原本一个写入需要同时完成主数据和索引数据的写入，需要通过分布式事务同时完成两分片操作。而优化后，由于无需同时写入索引数据，系统只需完成主数据分片的数据写入后就可返回。



通过上述优化，我们几乎将目前业务场景中所有的两阶段提交都优化为了一阶段提交，消除了系统中绝大部分的跨分片事务，避免了大量分布式事务产生的性能开销。

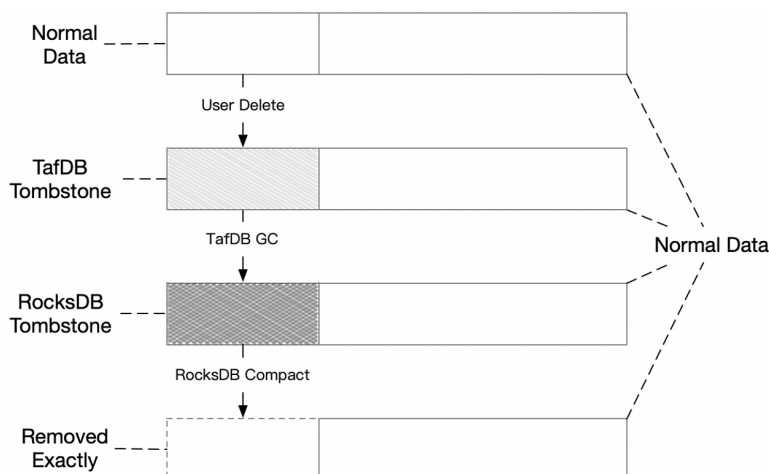
6.2 挑战二：在提供高性能写操作的同时，保证范围查询的性能

6.2.1 痛点

TafDB 使用 RocksDB 作为单机存储。RocksDB 使用 LSM-tree (Log Structured Merge Trees) 结构组织数据存储，其将删除视为一种特殊写入，即删除操作会插入一个墓碑 (Tombstone)，将 key 对应的旧数据标记为垃圾数据，最终通过 compaction 完成数据的异步删除。

为了保证事务的实现，TafDB 采用多版本的数据存储设计，与 RocksDB 类似，也使用标记删除的方式。被标记的垃圾数据最终会通过 GC (多版本垃圾回收) 机制异步完成其在 RocksDB 中的删除。不过如上所述，磁盘上数据的实际删除还要等待 RocksDB 的 compaction 完成。

"标记删除 + 异步清理"的方式会产生垃圾数据。由于存在 RocksDB & TafDB 两层标记删除，垃圾数据在系统中的滞留时间会进一步增加，进而增加垃圾数据量。由于单机消除垃圾数据能力有限，当系统中出现连续的删除时会产生一段垃圾数据区，这些垃圾会增加这段区域上范围查询过程中的比较次数，极大地影响范围查询的性能。

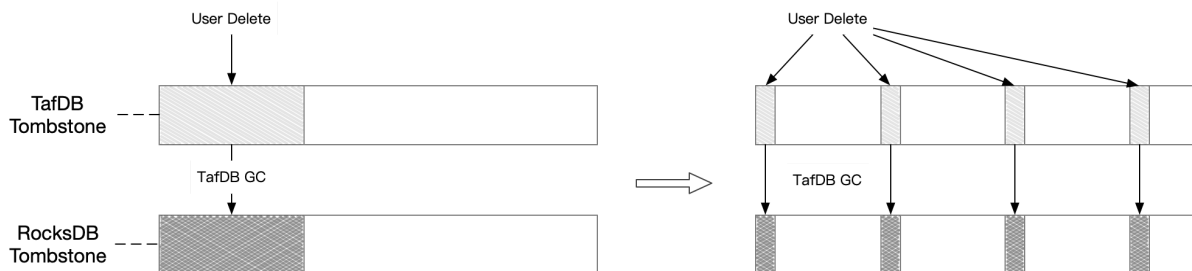


为了解决这个问题，即缩短连续垃圾数据区的长度。我们将数据删除的压力分散开，将垃圾数据区拆成多个小片；同时增强单点处理垃圾数据的能力，缓解其对范围查询的性能影响。

6.2.2 解决方案

针对这个问题，我们从三方面进行设计。Scale up 方面，提升系统处理单点数据 GC 和 compaction 的能力；Scale out 方面，尽可能将删除压力分散分解；防御反馈方面，力求在系统受到高压时，最大程度地保证系统稳定、缩小影响面并及时反馈，针对性地进行恢复。

- Scale up：重构系统 GC 模块，将单层全局 GC 扩展为多层次、特征化的 GC。我们根据业务场景和自身设计，采集了系统数据的多维度特征。在此基础上，GC 管理模块便可以针对不同的特征集设置不同层次的 GC 策略，以此保证各分片 GC 的差异性——该 GC 的及时 GC，不该 GC 降低 GC 频率节约资源。GC 管理模块会对各层次的 GC 策略进行组织和协调，防止多个策略间出现冲突或者产生无谓的资源消耗。另外我们实现了垃圾数据版本分布的精准探测，以减少 GC 时的大量数据扫描，有效提升 GC&compaction 的效率。
- Scale out：将用户操作产生的垃圾数据尽可能地分散在多个分片、多个 RocksDB 中，缓解单点的压力。虽然我们针对 GC 进行了大量的优化，但单点 GC&compaction 的能力一定是有上限的。因此我们考虑将用户产生垃圾数据的操作尽可能地分散，来提升系统应对该问题时的整体弹性。RocksDB 维度，我们会感知各个 RocksDB 上产生垃圾数据的压力，并在其压力过大时将部分分片迁移走，防止 RocksDB compaction 能力不足。分片维度，我们能感知用户范围性的数据删除或更新操作，并将其分散到多个分片，来减缓单分片的 GC 压力。如图所示，上述策略可以有效地将两层 Tombstone 分散开，避免大量连续垃圾数据对范围操作的影响，并通过并发处理提升系统整体的垃圾数据处理速度。



- 流控与反馈：对系统各方面的操作压力进行了多维度的统计，防止过载并及时进行针对性的恢复。以 list 请求为例：流控方面，我们对其进行并发度限制来保证系统不会过载。其中，通过感知不同 list 请求消耗时间和资源的，系统可以为其赋予不同的权重，来保证并发限制过程中系统资源仍能被充分的利用。反馈方面，当请求扫过的垃圾数据量过多或扫描过慢时触发对应的垃圾回收。

6.3 挑战三：消除数据流程的单点，提供极致的扩展性和可用性

6.3.1 痛点

元数据存储需要极高的扩展性来支持数据的大量存储和快速增长。在存储层，TafDB 通过基于 Range 分区的分片分裂、调度、回收机制提供了极致的扩展性；在接入层，无状态、可随意增减的 Proxy 也保证了极致的扩展性。

但仅仅这些是不够的，当今超大规模元数据场景设计需要面向千万级别以上的 QPS，当系统读写流程存在单点时，当前的硬件条件无法支撑这个级别的规模。TafDB 最初使用一个单点全局授时服务（TSO）提供单调递增的时间戳，确保分布式事务时序，这种方式可以达到百万级别的 QPS。为了满足更高的扩展性，我们通过各节点使用其本地时钟的方式解决了这个单点瓶颈。

6.3.2 解决方案

业界的时钟解决方案分为三大类：TSO（全局发号器）、HLC（混合逻辑时钟）和 TrueTime。全局发号器方案的问题就是上面提到的可用性和性能瓶颈；CockroachDB 采用了 HLC 方案，HLC 没有中心化的性能和可用性瓶颈，但是这需要和 DB 逻辑耦合，实现复杂度高；Google 的 Spanner 采用的是 TrueTime 方案，它是一个完美的方案，但是需要依赖专有硬件。

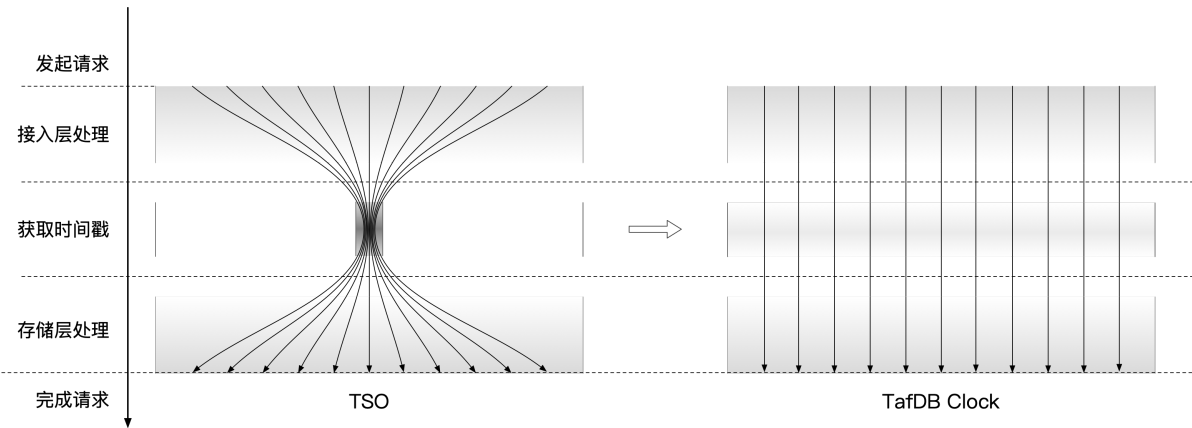
时钟解决方案对比：

	TSO	HLC	TrueTime
优点	- 全局一致时间 - 实现简单	无中心化的性能和HA瓶颈	- 全局一致时间 - 开发简化
缺点	- 时钟获取都涉及网络延时 - 跨AZ场景下性能损耗大 - 性能和可用性瓶颈	- 时钟和DB逻辑耦合 - 缺乏中心时钟，调试复杂度高	需要专有硬件

通过对业界时钟方案的对比，很容易想到使用 HLC 方案来解决中心式时钟的单点问题。但是如上所述，该方案在实现上比较复杂，存在不可知的系统性风险。

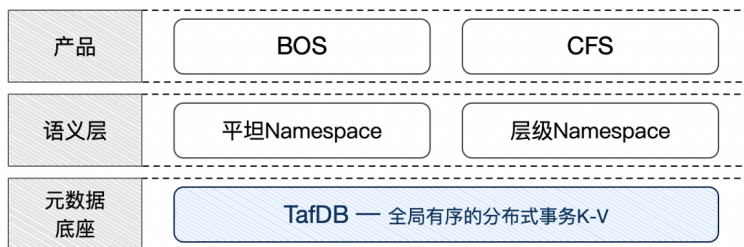
最终我们设计了 TafDB 独特的分布式时钟方案（TafDB Clock）：每个存储节点维护本地时钟服务。对于单分片事务，直接使用本地时钟；而对于跨分片事务，通过广播确保整体的因果序。由于 TafDB 中绝大多数事务都被优化为了单分片事务，该方案并不会引入广播产生的显著性能开销。

通过 TafDB Clock 我们消除了时钟的单点隐患，同时不增加额外的事务逻辑复杂度。如图所示，当使用 TSO 时，系统中所有的请求都需要挤在一起访问单点的时钟服务以获取时间戳，而单点是存在上限的，只能支撑百万 QPS 规模，并且一旦时钟服务异常就会影响集群全部的请求。而使用 TafDB Clock 方案时，所有的请求不再需要访问某个共同的节点，上述单点问题也就被消除了。



7. TafDB 应用效果

通过以上设计和优化，我们实现了一套功能完备，具备极致性能和扩展性的元数据存储系统。TafDB 满足了所有元数据存储的需求，统一了百度沧海·存储的元数据底座，简化了多套系统带来的复杂性。



7.1 TafDB 在文件系统 Namespace 存储的应用

百度沧海·文件存储 CFS 基于 TafDB 推出新一代 Namespace 架构，可以在 POSIX 语义层做到线性扩展，延迟方面写为 2ms，读延迟只需要百 us 级。这使得文件存储 CFS 不仅可以支持传统应用，作为传统业务上云的存储方案，也可以应用于最新的 AI 场景，满足海量文件规模处理的应用需求。

7.2 TafDB 在对象存储 Namespace 存储的应用

百度沧海·对象存储 BOS 基于 TafDB 推出新一代 Namespace 架构，单 Bucket 容量从 500 亿级别提升到万亿级别，可满足任意客户的容量需求，小文件延迟降低 42%，极大的提升了图片类业务的上传和下载用户体验。